

Spherical Decision Trees: A Short Geometric Note

Mathias Valla

May 13, 2026

Abstract

Classical decision trees split the covariate space with coordinate thresholds. This rectangular geometry is simple and effective, but it can be inefficient when the response is controlled by a localized region of the feature space, for example observations close to a prototype, a centroid, or a compact cluster. We study a small modification of recursive partitioning in which a node sends observations left or right according to whether they fall inside or outside a hypersphere. The method keeps the familiar tree architecture while replacing some one-dimensional threshold questions by distance-to-center questions. We describe the construction, give a simple geometric argument showing that distant sphere centers recover linear splits as a local limiting case, and summarize an exploratory benchmark against axis-aligned and oblique trees. The benchmark suggests a clear empirical pattern: spherical random forests obtained the highest mean balanced accuracy among the compared forest methods on the classification benchmark, although this advantage does not carry over generally to regression or to single trees. Single spherical trees appear most useful in low-dimensional nonlinear settings where the signal is naturally radial, localized, or cluster-like. We therefore view spherical splits not as a replacement for CART, but as a complementary local geometry for tree-based models.

1 Motivation

Decision trees are attractive because they are nonparametric, interpretable, and easy to ensemble. Their usual form, however, is tied to a strong geometric choice. At each internal node, CART-style trees select a feature index j and a threshold a , then split observations according to $x_j \leq a$ or $x_j > a$ [1]. Recursively, this produces rectangular cells.

Rectangular cells are natural when the signal is sparse and coordinate-aligned. They are less economical when the target depends on an area around a center. In classification, a class may correspond to a compact region; in regression, a response surface may change around a point, centroid, or prototype. An axis-aligned tree can approximate such a region, but it may need many cuts. A spherical split can isolate the same region directly.

Spherical splits can also be interpretable in a way that differs from oblique splits. An oblique rule asks whether a weighted linear combination of covariates exceeds a threshold. This is flexible, but once several variables are involved the resulting score can be difficult to read. A spherical rule asks whether an observation lies within radius r of a center c . When the covariates live in a meaningful metric space, this can be a natural local-neighborhood statement. For instance, in a classification problem with projected latitude and longitude as covariates, a split corresponds to a circular region around a geographical point. With an appropriate spatial projection or distance metric, the rule can be described as “within distance r of this location” rather than as an abstract linear score.

2 Spherical splits

Let $S \subseteq \{1, \dots, p\}$ be the set of features used at a node. A spherical split is defined by a center $c \in \mathbb{R}^{|S|}$ and a radius $r \geq 0$:

$$\|x_S - c\|_2^2 \leq r^2.$$

Observations satisfying this inequality are sent to one child and the remaining observations to the other child. The pair (c, r) is selected to maximize the same impurity decrease used in ordinary tree induction, such as Gini decrease for classification or variance reduction for regression.

In the implementation studied here, candidate centers are generated from a finite mixture of rules: global centers, target-conditional centers, observed points, pairwise midpoints, local Gaussian perturbations, and farther radial perturbations. For each candidate center, candidate radii are induced by the Euclidean distances from the center to node samples. This preserves the usual finite split search structure: the geometry changes, but the recursive optimization problem remains a sequence of impurity comparisons.

Figure 1 compares three split geometries on several two-dimensional examples. The learned spherical rule at each internal node is a circle; the terminal regions arise from recursive inside/outside decisions.

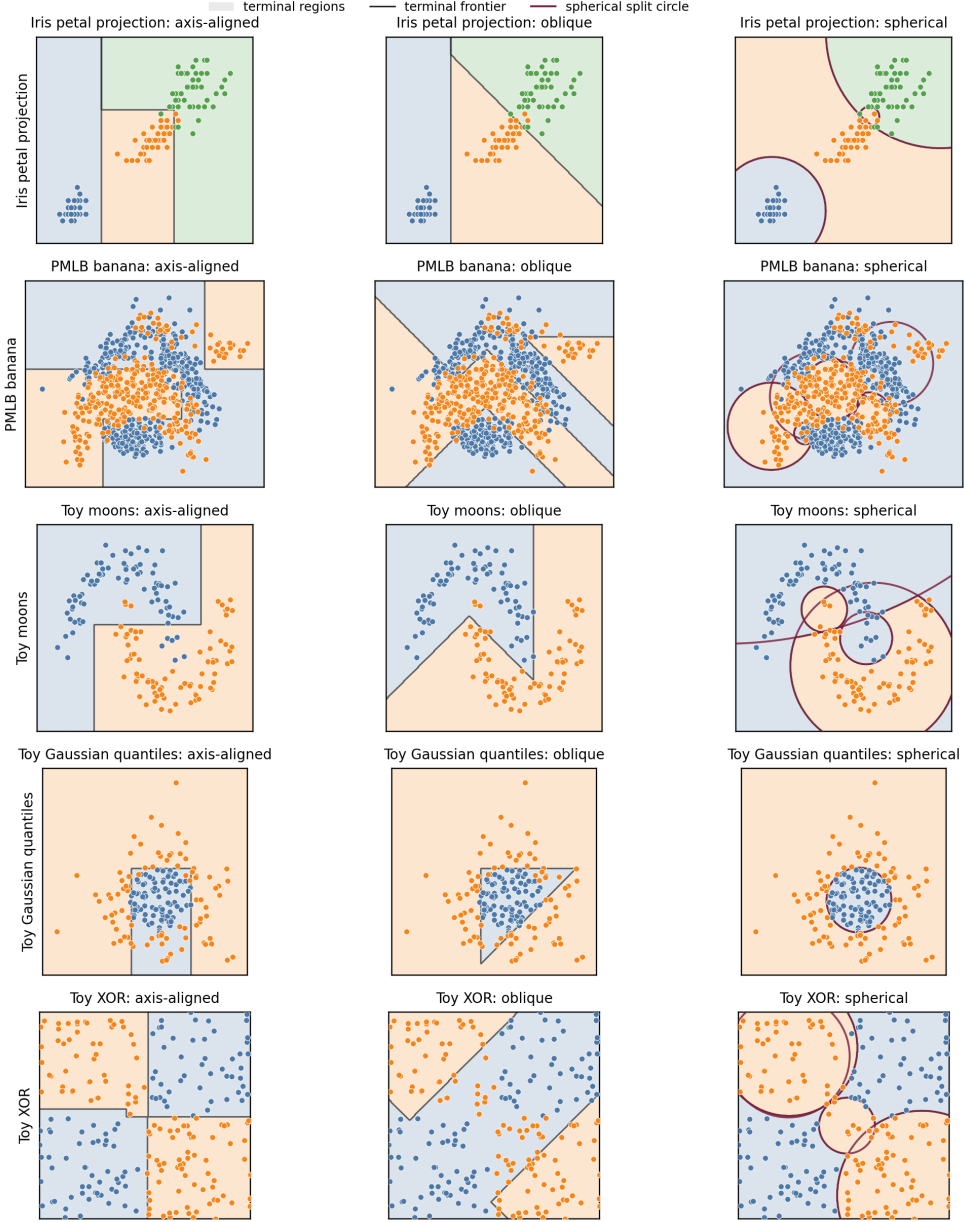


Figure 1: Comparison of two-dimensional decision regions learned by shallow axis-aligned, oblique, and spherical trees. Each row is a dataset and each column is a splitting geometry. Axis-aligned trees form rectangular recursive partitions, oblique trees use linear-combination thresholds, and spherical trees form regions by recursive inside or outside decisions around learned circles. Black curves are terminal decision frontiers; burgundy circles in the spherical panels are internal split boundaries. The figure illustrates when spherical geometry is economical, for example in localized or curved class structures, and also shows that greedy induction can remain imperfect on interactions such as XOR. All panels use standardized coordinates and depth-three trees for readability.

3 Two useful geometric facts

The first useful case is local radial structure. If the target changes when x enters a ball around c , then a single spherical split can represent that event. A rectangular tree, by contrast, must approximate the ball by many coordinate cuts. This is an approximation-economy argument rather than a claim that spherical trees are universally more expressive.

The second useful case is the limiting connection to linear splits. Let $u \in \mathbb{R}^p$ satisfy $\|u\|_2 = 1$, let $R > 0$, set $c = Ru$, and choose

$$r_R^2 = R^2 - 2Rt.$$

Then

$$\begin{aligned} \|x - Ru\|_2^2 \leq r_R^2 &\iff \|x\|_2^2 - 2Ru^\top x + R^2 \leq R^2 - 2Rt \\ &\iff u^\top x \geq t + \frac{\|x\|_2^2}{2R}. \end{aligned}$$

On any bounded region $\|x\|_2 \leq B$, the final term is at most $B^2/(2R)$. Thus the spherical split converges locally to the halfspace $u^\top x \geq t$ as the center is moved far away. Axis-aligned splits are recovered by taking u equal to a coordinate vector. In finite samples this is only a local approximation, but it motivates center-generation schemes that include far-away candidates.

4 Exploratory benchmark

We implemented spherical trees and spherical random forests in **treeple** and compared them with axis-aligned CART and random forests, as well as oblique trees and forests. The quantitative benchmark used complete PMLB datasets that ran without errors; the toy datasets in Figure 1 are used only for qualitative two-dimensional illustration. Features were imputed and standardized. The main benchmark used three cross-validation folds, 50 trees for forests, 500 spherical center candidates per node, all data-induced radii for each candidate center, validation-selected pruning for single trees, and unpruned forests. Large datasets were capped at 1000 sampled observations. The final comparison contained 249 complete datasets and 38 skipped datasets or fetch/model errors. The comparison is descriptive: it is intended to characterize regimes and failure modes, not to prove dominance.

The salient empirical result is the classification performance of spherical forests. In this exploratory benchmark, spherical random forests obtain the best mean classification score among the compared methods, outperforming both ordinary and oblique random forests, although at a much larger computational cost. This advantage is not observed generally for regression, where ordinary and oblique random forests remain stronger. Nor is it observed on average for single trees. The evidence therefore points to a specific use case: spherical split candidates can be valuable when their high-variance behavior is stabilized by ensembling, especially in classification problems with localized, radial, or cluster-like structure.

Figure 2 gives a compact view of the empirical regime map. It should be read cautiously: the number of rows and predictors alone does not fully determine when spherical splits help. The missing variable is likely geometric: whether the target depends on local neighborhoods, clusters, or distance-to-prototype effects.

5 Limitations

Spherical trees introduce several costs. Training is slower because each node must evaluate multiple candidate centers and radii. If a node contains n_m samples, s active features, and C candidate

Table 1: Mean benchmark performance and fit time over complete datasets. Primary score is balanced accuracy for classification and R^2 for regression.

Task	Model	Datasets	Score	Fit time (s)
Classification	Spherical random forest	132	0.765	9.850
Classification	Random forest	132	0.738	0.137
Classification	Oblique random forest	132	0.737	0.154
Classification	CART	132	0.722	0.074
Classification	Oblique tree	132	0.706	0.136
Classification	Spherical tree	132	0.688	4.132
Regression	Random forest	117	0.717	0.384
Regression	Oblique random forest	117	0.700	0.434
Regression	Spherical random forest	117	0.665	20.533
Regression	CART	117	0.521	0.078
Regression	Oblique tree	117	0.391	0.093
Regression	Spherical tree	117	0.262	7.516

centers, computing distances is roughly $O(Cn_ms)$ and evaluating sorted candidate radii can add $O(Cn_m \log n_m)$. This is much more expensive than scanning coordinate thresholds.

The method also depends on the metric. In our experiments, predictors were standardized before fitting. Without scaling, a spherical split is dominated by high-variance coordinates. In high-dimensional spaces, Euclidean distances can become less informative, and far-away centers may make splits nearly linear without solving the underlying greedy-search problem.

The XOR example is instructive. In principle, two far-away circles can locally approximate the two axis-aligned cuts needed for XOR. A greedy impurity criterion, however, may reject those cuts at the root because each individual axis split has little immediate impurity gain. This does not contradict the linear-limit argument. It shows that spherical trees inherit the myopia of ordinary greedy tree induction. A depth-two lookahead or delayed-gain criterion would be needed to systematically recover such interactions. For single trees, the more favorable low-dimensional cases are nonlinear settings with compact or curved decision regions, such as moon-shaped classes, the PMLB banana classification problem, or radial Gaussian quantiles.

6 Conclusion

Spherical splits provide a simple geometric extension of recursive partitioning. They are most appealing when the target is expected to depend on membership in an area of the covariate space, especially around a center, prototype, cluster, or spatial location. Because distant centers recover linear frontiers locally, the same mechanism can also approximate ordinary hyperplane splits. The main empirical finding is that spherical forests improved mean classification performance in the benchmark studied here. This improvement is not general for regression, and single spherical trees are competitive mainly in specific low-dimensional nonlinear regimes.

The natural next step is a hybrid tree that chooses among axis-aligned, oblique, and spherical split candidates at each node, possibly with a small lookahead window. Such a model would keep the interpretability of recursive partitioning while allowing the geometry of each decision to adapt locally.

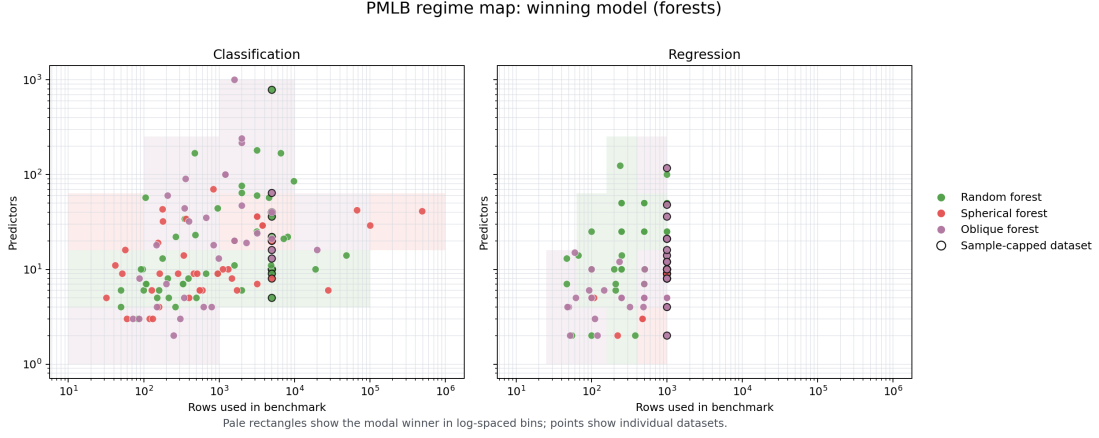


Figure 2: Winning forest model across benchmark datasets as a function of sample size and feature count. The pattern is exploratory and does not provide a deterministic rule, but it suggests that spherical forests are a useful complement in some classification regimes.

References

- [1] Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. *Classification and Regression Trees*. Wadsworth, 1984.
- [2] Breiman, L. Random forests. *Machine Learning*, 45:5–32, 2001.
- [3] Murthy, S. K., Kasif, S., and Salzberg, S. A system for induction of oblique decision trees. *Journal of Artificial Intelligence Research*, 2:1–32, 1994.
- [4] Olson, R. S., La Cava, W., Orzechowski, P., Urbanowicz, R. J., and Moore, J. H. PMLB: a large benchmark suite for machine learning evaluation and comparison. *BioData Mining*, 10:36, 2017.
- [5] Pedregosa, F. et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.